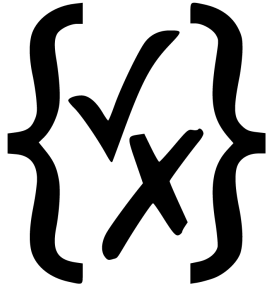




Google Summer of Code



{✓} JSON
Schema

Build more. Break less. Empower others.

JSON Schema

Google Summer of Code 2026

[Index](#)

Sr. No.	Title	Page No.
1.	Project and Student Details	2
2.	About Me and Open Source	3–7
3.	Contributions and community presence	6–9
4.	Project Details, Qualification Tasks and Expected Outcomes	10–18
5.	Timeline and Future Work	19-20

Personal Information

Name: Rahul Singh

Email: rs4515080@gmail.com

Github: [Kuzuri247](#)

Twitter(X): [Kuzuri247](#)

Proficient Language: English and Hindi

Location: Mumbai, Maharashtra

Time Zone: Indian Standard Time (IST) (UTC +5:30)

Phone number: +91 883 083 5763

Institution: Vidyavardhini's College of Engineering and Tech

Program: Bachelor of Engineering in Computer Science

Year of Study: Final year student (expected 2026)

Project Information

Name: Enhanced Interaction and Navigation in JSON Schema

Length: 90 hours

Mentor: [Agnivesh Chaubey](#)

Assisting Mentor: [Jagpreet Rahi](#)

Description: Further improvement and enhancement of the selected JSON Schema Studio project after receiving and strong feedback from the community. This year's project focuses on improving interactivity, navigation and rendering correctness by enhancing dependency ordering, editor synchronization, collision handling and user driven exploration features, including collapsible nodes, bidirectional editing and focused graph navigation and visualization and other enhancements leading to a perfect tool for developers.

About Me

When I was exposed to software engineering I got to know about open source software development. I started learning web development back in 5th semester of my college. I was confused that if the standards of a developer are so high today then how can I get an exposure to large codebases as a fresher. As I had done only data structures and algorithms before I had no idea of software development but then little by little I started trying out different things and realized that there is enough for everyone in this community and started learning things like React. I started with React directly and learned JavaScript in parallel as concepts became clearer through practice.

After a few months when I got used to javascript and other technologies in web dev and things started to click. Then I started looking for some projects. I came across a startup project, Sanity Gaming (now Toornify), and reached out to contribute. I worked there on various things from frontend, backend to various APIs and implemented features and reviewed code. I also made my own projects while learning and I had also applied to Gsoc last year but didn't get in because being new to the project I applied which was a devops related program in which my proficiency was not upto the mark.

Then I started making my own personal projects and tried to turn them into products. Like a voice AI app made by using Vapi's voice agents and completed with responsive cool UI. I also made my personal portfolio which is still incomplete because of the projects that are currently still being made by me.

My Plator project turned out pretty good and grabbed the attention of real users. Serving its purpose while I will still keep it improving and adding features to it. The projects which are currently still in the making are the projects which I am making with reference to the Greek pantheon of gods resembling different dev tools. 'Athena' is a web-based coding IDE designed to support multiple projects simultaneously, allowing developers to write and manage code directly in the browser without heavy local resource consumption. Another project is "Hades" an API platform which will be focused on making APIs and core features instead of being bloated like Postman. I previously used React Flow to build the visual canvas for an n8n-inspired project called 'Hephaestus'. While searching for GSoC opportunities, I learned about JSON Schema through Karan Palan's journey. Realizing the JSON Schema Studio project heavily utilized React Flow, I knew it was the perfect opportunity to leverage my existing skills while deepening my expertise in visual node-based editors.

All things considered I am also into gaming and attending tech events near me also joining tech spaces on X through which I have found amazing people who belong to different fields.

Programming Languages: JS/TS, C++, Rust, Go

Libraries and Frameworks: React, NextJS, Tailwindcss, Motion, shadcn, MUI, Carbon Design, Zustand, NodeJS, ExpressJS

Databases and ORM: Postgres, Prisma and Mongo

Tools and managers: Git, Github, Gitlab, VSCode, Antigravity, Postman agent, Docker, Vercel, Netlify, Imagekit, Cloudflare, npm, pnpm, yarn and bun

Open Source Collaborations

I have contributed to various open source projects with different aspects and tech used from devops to management platforms to my own personal projects. I know how to manage different branches and collaborate with other people and I got to know about this by contributing as a contributor and being a maintainer as well.

Some contributions I made include:

1. OpenCost UI revamp (latest for LFX)

For the LFX Mentorship program, I worked on the OpenCost UI revamp project where the goal was to build a new Assets page with graph visualizations using Carbon Design as the primary UI library. With no prior experience in Carbon Design and only ten days, I converted the entire demo project's component system from MUI to Carbon Design, not just the Assets page but across the full codebase. I was not selected, but the experience taught me that technical output alone is not the deciding factor in open source. Sustained community presence and understanding a project's long-term direction matter equally. That lesson directly shaped how I approached my JSON Schema contributions: reading the codebase thoroughly, raising well-reasoned issues before PRs, and engaging with maintainers meaningfully before writing proposal code.

2. Kubebuilder (Last year's Gsoc project)

Kubebuilder is the project that taught me how to use every aspect of Git and work effectively across branches in a large

codebase. As Golang and DevOps were new paradigms for me at the time, I focused my contributions on areas where I could deliver immediate value, such as the CI/CD pipeline, while actively learning the broader architecture under the maintainer's guidance. I contributed to the pipeline and other small things. There were strict rules and guidelines for contributing to the project which taught me how to use Git effectively and how work in a big codebase is done. It was a great learning experience.

3. Sanity Gaming

This project taught me about handling a codebase for the first time after I had just learned NextJS. Here I contributed and reviewed other people's code and worked collaboratively with other maintainers discussing tasks and working out on different features. This project taught me how to work in a team and go back and forth in the process. This project later got closed source.

I discussed my various personal projects earlier. I started pretty late but still the learnings compounded and today it's 1.5 years of learning and building.

I have worked on my portfolio which is still in making and I will update it once the projects are reaching a certain level of stage where I can call them products not just projects.

Source of Motivation

I got to know about JSON Schema from Karan Palan. He got selected in Gsoc last year and I got to know about the Org

through his podcast with Harkirat Singh. I also stumbled across Utkarsh Tiwari and Dhairya Majmudar's accounts. When I explored I got to know that there are various projects where I can contribute.

I explored the spec and website repos and got a basic about the organization and started with small contributions where I got to about the system of raising issue before PRs are preferred in the organization which I also is a good way to get good contributions rather than random AI made contributions where a person doesn't know about the thing he pushed. I explored the organization's Gsoc page where the projects that were selected last are mentioned and from there I got to know that Jagpreet, who got selected in this project last year had joined the triage team and works with the org. Which is my goal as well in which I want to be established as a maintainer and contributor of such a large org.

I was using Reactflow for the n8n project but the project got abandoned for no reason. I am planning to resume that project and work on it and this became a big reason I wanted to get into the studio project. Because when I looked closely into the project being new and my contributions will play a huge role in shaping the project got me excited as there are many things that can be improved in the project currently.

I prioritized meaningful, well-reasoned contributions over volume, raising issues before PRs as preferred by the organization.

Contributions and Issues raised in the repositories till now more on the way:

Issues and Pull Requests:

1. Studio-json-schema: Some issues and pull requests in the main repository

Sr. No.	Issues and their related pull requests issues information
1.	#110 : Node highlighting issue > #111 : Node gets highlight or selected first then content table opens after the second click which is the more sensible and common practice
2.	#112 : Minimap with highlighting > #113 : Introducing a minimap for the canvas for easy maneuvering and overview of the map
3.	#114 : Proper control panel > #115 : Fixing the controls with proper theme and also redefining the searchbar
4.	#116 : Package dependency > #117 : Fixing the browser package dependency enabling smoother build and setup
5.	#125 : Custom Cursor design > #131 : Redesigning the default cursor with some custom theme for better user interaction
n.	I will raise more issues and prs which will make a difference

2. Website: Some minor contributions in the JSON Schema website

This repository was being spammed with prs so I made a few

1.	#2080 : Fixing the image aspect and icons in the landing page's community section
2.	#2095 : Revamping the tools sections sidebar
3.	#2083 : Fixing the card image and header text format

There were some other PRs as well but they were small and didn't have much value so they were closed by the maintainers.

Community Presence

Now I have been in this community since January and also helped a few folks by telling them about the community and had a chat with them on social platforms. I also had a discussion about the version and update of the website from the old NextJs version to the latest version and people and maintainers showed great interest in those discussions.

I had a talk with Greg a few times and I also felt the ecosystem observability project pretty similar to the Opencost's LFX project in which metrics are to be shown for the JSON Schema ecosystem with the goal to improve its observability and visibility for new developers and students getting involved with the open source projects and solve various problems.

Other than that I have talked to many people in the community along with the mentors of this project Agnivesh and Jagpreet and cleared my doubts about the project and proposal submission.

Overall JSON Schema has a fantastic community who help each other when needed.

Project Details

The project aims on the graphical representation of JSON and other format schemas with interactive visualization helping developers to understand the core infrastructure of big codebases and debug complex schemas. The project was selected last year in GSoc-2025 where the mentee now assisting mentor of the project Jagpreet Rahi who made the foundation of the project and this year it is more focused on polishing and improving the existing project with more meaningful additions and features implemented in the project. Like a minimap for example, when there are huge structures and patterns within a huge codebase there is a lot of complex data generated in the process which leads to large files which can be intertwined and aligned together making a huge entity. To tackle such situations a minimap with proper coverage and node highlight and other features.

Qualification Tasks

I have made a list of both the qualification tasks and the rest outcomes that are expected to be the area of focus in the Gsoc-2026 [repo](#) in which all the things are mentioned as requested. Now looking into tasks we can proceed to draw some key notes for the handler of Dependent Schema and topological sorting of the dependencies. I understood the qualification tasks and I have figured a way out to handle those which are mentioned below.

Qualification Task 1: `dependentSchemas` Keyword Handler

So basically `dependentSchemas` keyword applies a subschema to the entire object when the named property is present.

```
{
  "dependentSchemas": {
    "credit_card": {
      "required": ["billing_address"]
    }
  }
}
```

Meaning if the `credit_card` exists in the instance the entire object must have the billing address. Without a proper handler this conditional structural relation is not visible in the graph. Which leads to the user missing the `{"credit_card": "1234"}` context when the `{"name": "Alice"}` is valid.

Hyperjump compilation makes the difference here where a correct handler is separated from the naive one. In the existing `keywordHandlerMap` in `processAST` the properties handler does

```
for (const [key, value] of Object.entries(keywordValue as string))
```

Since the hyperjump properties compile function uses `asyncCollectObject`, producing `{propertyName: schemaUri}`.

`dependentSchemas` is different as it compiles `func asyncCollectArray` which produces an array of `[propertyName, schemaUri]` tuples instead of a plain object. This means

Object.entries() would fail silently and iterate incorrectly. The handler must cast keywordValue as [string, string] and user array

```
"https://json-schema.org/keyword/dependentSchemas": (
  ast, keywordValue, nodes, edges, parentId, nodeDepth, renderedNodes
) => {
  const entries = keywordValue as [string, string][];
  const propertyNames: string[] = [];

  for (const [propertyName, schemaUri] of entries) {
    propertyNames.push(propertyName);
    processAST({
      ast, schemaUri, nodes, edges, parentId,
      renderedNodes, childId: propertyName,
      nodeTitle: `dependentSchemas["${propertyName}"]`,
      nodeDepth
    });
  }
  return { key: "dependentSchemas", data: { value: propertyNames } };
},
```

leafNode: true is deliberately not returned. Omitting it ensures generateSourceHandles creates one outgoing handle per property name, producing the correct visual edges in the graph. Additionally, dependentSchemas is already listed in objectKeywords inside inferSchemaType.ts, so nodes with this keyword are correctly colored as object-type schemas with zero changes required outside processAST.ts.

Qualification Task 2: Topological Sorting of \$defs dependencies

The current sortAST moves the definitions keyword handler to the front of the AST keyword list for a schema node. It does not sort the \$defs entries themselves by dependency order.

```
{
  "$defs": {
    "Address": { "$ref": "#/$defs/Country" },
    "Country": { "type": "string" }
  }
}
```

The hyperjump AST processes \$defs values in declaration order. Address comes first. When processAST visits Address, it finds a \$ref pointing to the Country Uri. Since the country has been added to the renderedNodes the \$refs handler calls processAST recursively on Country but now with nodeDepth inherited from the Address context not the the \$defs context. Country ends up being a child instead of sibling of Address

Three concrete rendering bugs result:

- => Country gets the wrong nodedepth it because it appears to be one level deeper than it should be
- => Country appears nested under Address instead of at the same \$defs peer level
- => For recursive references **A > B > A**, the renderedNodes guard prevents infinite loop but leaves nodes various incorrect positions

After evaluating all viable approaches, Kahn's BFS is the most suitable for this use case, as the comparison below shows.

Method	Cycle Detection	Time Complexity	Stack Overflow Risk	Output Order	Fit for \$defs
Kahn's BFS	Implicit (leftover nodes)	$O(V+E)$	None (iterative)	Breadth-first (intuitive for dagre)	Best
Recursive DFS	Needs visiting set	$O(V+E)$	Yes (deep chains)	Post-order (correct)	Good
Iterative DFS	Needs visiting flag	$O(V+E)$	None (explicit stack)	Post-order (correct)	Good
Source Removal	Implicit (empty remainder)	$O(V^2)$	None	Declaration-biased	Inefficient
Matrix Squaring	N/A	$O(V^3 \log V)$	None	By rank	Overkill
Insertion Sort	None — causes infinite loop	$O(V^2)$	None	Unstable partial order	Broken

As we see here by comparison and from what I configured I found out that Kahn's algorithm is the possible way to do this.

Kahn's algorithm is better here because of the cycle detection being implicit which is the leftover nodes are cyclic \$defs entries that the existing renderedNodes guard in processAST already handles as backedges. DFS needs additional visits to figure that out.

BFS matches the dagre assignment and the output leaf definition by Kahn's are level by level. Since the dagre assigns rank by node insertion order for the same level this produces the most visually logical LR output layout without second layout pass.

Kahn's algo handles depth safely instead of overflowing JS call stack on pathological schema.

Expected Outcomes

1) Topological Sorting \$defs rendering

The topoSortDefs function above is called inside the \$defs handler before the processAST loop, replacing the raw value array with the sorted one. Zero changes to any other file.

2) Validation Error Navigation

In the current state, the Monaco editor uses jsonc-parser's parseTree and findNodeAtLocation for parsing, and Hyperjump compile errors include JSON Pointer fragments. The entire bridge between the error location and the editor line already exists and jsonc-parser is already imported in MonacoEditor.tsx meaning no new dependency is needed.

Implementations steps:

- Extract JSON pointer from Hyperjump error
- jsonc-parser.findNodeAtLocation resolves to line/column offset

- Make the error list clickable => replacing the static error div in the validation status bar with an interactive list

Data Flow : Hyperjump error > JSON pointer > jsonc-parser offset > Monaco marker > clickable error item > revealPositionInCenter

3) Edge Collision Resolution

The resolve Collisions handles only bounding box overlaps via vertical push and the edge crossing are entirely unaddressed.

- Orthogonal edge routing in which all the smoothstep edges will be replaced by a custom component using getSmoothPath with border radius zero to produce right angle routing which naturally reduces visual tangling
- Parallel edges with proper offset margin should reduce the overlapping and collision
- Increasing ranksep and nodesep in dagre config will give us more routing room for edges

4) Edge Based Navigation Control

Edge selection only triggers color highlighting which is fixed by me in one of my PR. Currently no navigation actions exist on edges beyond color highlighting.

- Rendering of a smooth bezier curve showing a floating point at edge midpoint on hover

- On “<- Source” click calls the center position function with some transition and with “Target->” same for the target node
- Panel background at 80% opacity for visual coherence

All edges need type navigable Edge type defined

5) Handle Misalignment on Live Edits

React Flow node uses `uselayout` to handle offsets by reading DOM `offsetParent` rects. When Monaco fires `onChange`, a re-parse generates new `sourceHandles` arrays but if the data reference doesn't trigger a re-measure, handles stays at a stale position.

- Calling `useUpdateNodeInternals()` from `xyflow` after schema edits that add/remove source handles. This is the `reactFlow`-idiomatic way to force handle functions recalculation and is the cleanest single fix
- Using `uselayout` with `resize observer` on the node container firing the node's DOM dimensions change not just React render cycles

6) Direct Visualization of Multi-types schemas

Currently all the other types are leading to losing of the color information and array property is missing

- Adding `Array` as a node datatype returning a “multitype” category with the full type array

- Use of gradient segments and border colors to separate the nodes and applying badges to them for visual identification

7) File Upload Support

Implementing a UI dropdown box for drag-n-drop alongwith upload window option as well. Also toast notifications for different actions

8) Downloadable Visualization Output

With the help of html-to-image we can download the images of the graphs from the canvas and with help of blob we could download editor content as well.

9) Minimap Integration

This is my personal suggestion that an interactive minimap should be made for the project which is more than the minimap provided natively and has custom features like highlighting shifting and more so that the user gets an enhanced experience while navigating through huge structures.

All qualification tasks and outcomes expected from the internship are covered now we can discuss the timeline and how the tasks will be planned and divided.

Timeline

Phase	Time Period	Tasks
Pre GSoC / Proposal Evaluation	Apr 6 – Apr 26	Gather ideas and research deeply on how we can improve the platform that will enhance user experience with all the planning.
Community Bonding Period	May 8 – Jun 1	I will discuss all the planning and research with mentors and gather opinions for the ideas and my thinking with other members of JSON Schema. My final year semester exams will be happening during the early period but I will manage that accordingly with my time schedule
Coding Period Begins	Jun 2 – Jun 22	Executing the implementation of the dependentSchemas keyword handler and topological \$defs sorting. Integrating validation error navigation via Monaco markers. Resolving handle misalignment during live schema edits. Implementing edge collision resolution and orthogonal routing. Establishing sophisticated edge-based navigation controls

Midterm Evaluation	Jun 22 – Jun 26	Submit midterm evaluation report covering all features completed in the first coding phase
Coding Period Continues	Jun 27 – Jul 26	Multi-type schema visualization · File upload support with drag-and-drop. Downloadable PNG/SVG output. Minimap integration. Final bug fixes and end-to-end testing.
Final Evaluation	Jul 26 – Aug 3	Final project submission with all code, tests and documentation
Post GSoC Period	Aug 3 – Aug 10	Write GSoC wrap-up blog post for the JSON Schema community and maintain merged features. Plan next contribution cycle as an ongoing maintainer

I tried to make this timeline with the information provided in the Gsoc page covering each part of the program. I divided all the tasks according to their priorities and aspects in the project.

Post Gsoc Plan

My commitment to the JSON Schema organization extends well beyond the official GSoC timeline. I plan to remain an active maintainer of the JSON Schema Studio, ensuring that the

features implemented such as the topological sorting, validation navigation, and edge-based controls are robustly maintained and updated alongside the evolution of the JSON Schema specification.

I intend to tackle the stretch goals identified during the proposal phase, including the implementation of more advanced visualization patterns for complex conditional schemas and deepening the bidirectional editing capabilities between the code editor and the visual canvas.

Furthermore, I am dedicated to fostering community growth by documenting my journey and creating guides to help new contributors navigate the project codebase. Following the example of previous successful mentees, I will stay active in the organization's Slack and GitHub discussions to provide guidance and review contributions from the next generation of students.

Availability

I would devote a fixed number of hours per week for the project. During my final year exams in May, I will dedicate at least 1–2 hours per day to community discussion, mentor sync, and design documentation. From June onwards I will commit 30 hours per week considering the planning and execution of the project.

I am fully committed to delivering every outcome in this proposal. Every section has been written by me with direct reference to the project codebase.